

Belegvorstellung

Belegbeschreibung

Modul: B42 Softwareengineering und Softwarearchitekturen

Studiengang: Informatik in Kultur und Gesundheit

Semester: Sommersemester 26

Beschreibung

Ziel des Belegs ist es, eine RESTful Micro-Service-Architektur in Gruppen zu entwickeln. Die Anwendung modelliert ein Online-Casino, bei dem User, über bereitgestellte APIs, Accounts anlegen und risikofrei an den bereitgestellten Spielen teilnehmen können.

Begründung

Die typischen Spiele eines Casinos haben eine zumutbar hohe Komplexität und einen überschaubaren Umfang, wodurch deren Modellierung eine sehr gute Übung für aufstrebende Entwickler darstellt. Gleichzeitig sind sie realitätsbezogen, dadurch greifbar und auch interessant. Außerdem lassen sich die Spielprozesse und übliche Spielstrategien seitens Spieler und Haus leicht recherchieren.

Ziel

Das Glücksspiel soll natürlich nicht durch dieses Projekt glorifiziert werden! Im Gegenteil, Ziel ist es, die Gefahren aufzuzeigen, indem statistisch Gewinne und Verluste, sowie Gewinnchancen dargestellt werden.

Herausforderung

RESTful APIs liefern ad-hoc Representationen von Datenzuständen, wobei die Kommunikation selbst zustandslos ist. Das bedeutet, dass eine Anfrage gegen eine solche API in sich schlüssig ist und keinen Kontextbezug zu anderen parallelen oder historischen Anfragen herstellen kann. RESTful APIs dürfen aber zum Erfüllen ihres Use-Cases sich selbst gegenseitig miteinander beiziehen und Unterabfragen versenden. Die Wahl der zu modellierenden Spiele ist aus zuvor genannten Gründen eingeschränkt. Des Weiteren werden alle Spiele ausschließlich als Einzelspielerspiele modelliert und so, dass jedes Spiel nach einer Runde (einem API-Call) endet.

Services

Die Anwendung soll aus drei Micro-Services bestehen. Einem Banking-Service und zwei Spiel-Services, "Roulette" und "Slotmachine". Jeder der drei Services soll in einem eigenen Docker-Container laufen.

Zusätzlich hat jeder der drei Services eine Anbindung zu einer eigenen Datenbank, welche ebenfalls in separaten Containern laufen sollen. Die Gesamtanwendung besteht somit aus sechs Containern.

Die drei Kern-Services sind intern nach dem Muster einer "Model-View-Controller"-Anwendung aufgebaut. Der Banking-Service folgt der Vertikal Slice Architektur (Logische Layer, einschließlich "Model"-Layer, "View"-Layer und "Controller"-Layer, die alle jeweils nach den Sub-Domänen ge-sliced sind).

Die Domäne des Banking-Service ist nach "User", "Transaction" und "Stat" aufteilt.

Die Spiel-Services sind weniger streng nach der Geschichteten Architektur aufzubauen.

"Views" sind Objekte für die äußere Darstellung (JSON) im Sinne von REST, anderes als im klassischen MVC-Ansatz, indem "Views" Webdokumente abbilden.

Beispiel: Service-Struktur (MVC+Vertikal-Slice):

```
1  ...
2  \Controller
3      |_ \User
4          |_ ...
5          |_ ...
6          ...
7      |_ \Transaction
8      |_ \Stat
9  \Handler
10     |_ \User
11     |_ \Transaction
12     |_ \Stats
13 \View
14 \Repository
15 \Model
16 ...
```

Die Subdomäne "Game" beschäftigt sich mit der Spiellogik, wobei "Info" sich mit den Spielregeln und Gewinnchancen beschäftigt und "Stats" deckt die Gewinn- und Verluststatistiken, sowie die Spielhistorie ab.

Verpflichtender Tech-Stack

- Java (Programmiersprache)
- Java Spring Boot (Framework)
- Maven (Build-Tool)
- JUnit (Unit-Testing)
- Mockito (Unit-Testing)
- PostGre (Datenbank)
- Docker (Containerisation)
- Swagger (API-Dokumentation)
- PlantUML (Architektur-Dokumentation)

Abgabe und Dokumentation

Abgegeben wird:

- der gesamte Code in einer `.zip` Datei

- 3 Docker-Container-Images der drei Services als `.tar` Dateien, gemeinsam verpackt in einer `.zip` - Datei
- 4 PlantUml **Komponentendiagramme**
 - Diagramm zur **Darstellung des Gesamtsystems** (einschließlich Datenbank-Container)
 - Diagramme zur **Darstellung des internen Aufbaus** der einzelnen Services (ohne Datenbank-Container)

Beispiel: Erstellen einer Image-Datei

```
1 docker save -o mein-image.tar mein-image:latest
```

Anforderungen

⚠ Achtung:

Mit den Funktionalen Anforderungen wird auch Mitdenken vorausgesetzt. Einige Sub-Anforderungen ergeben sich implizit.

Nicht alles ist vorgekaut!

Funktionale Anforderungen

folgende Banking-Service-APIs sollen über die Anwendung nach Außen bereitgestellt werden:

- GET `http://localhost:8080/casino/bank/api/user/{id}`

- **Request Body:** nicht nötig
- **Response Types (& Bodys):**

- Ok - 200 :

```
{ }
```

```
1 {
2     "id": ..., //concrete user (see api path)
3     "first_name": ...,
4     "last_name": ...,
5     "balance": ... //current finances
6 }
```

- Not Found - 404

- GET `http://localhost:8080/casino/bank/api/users`

- **Request Body:** nicht nötig
- **Response Types (& Bodys):**

- Ok - 200 :

```
{ }
```

```
1 [
2     {
3         "id": ..., //some user
4         "first_name": ...,
5         "last_name": ...,
6         "balance": ... //current finances
```

```

7      },
8      {
9          "id": ..., //some other user
10         "first_name": ...,
11         "last_name": ...,
12         "balance": ...
13     },
14     ...
15 ]

```

oder

```
{ }
```

```
1 [] //empty array (initial system state)
```

- POST http://localhost:8080/casino/bank/api/user

- **Request Body:**

```
{ }
```

```

1 {
2     "first_name": ...,
3     "last_name": ...
4 }

```

- **Response Types (& Bodys):**

- OK - 201 :

```
{ }
```

```

1 {
2     "id": ..., //auto generated and given id
3     "first_name": ...,
4     "last_name": ...,
5     "balance": 0 //initial balance
6 }

```

- Bad Request - 400

- PUT http://localhost:8080/casino/bank/api/user/{user_id}

- **Request Body:**

```
{ }
```

```

1 {
2     "first_name": ...,
3     "last_name": ...
4 }

```

- **Response Types (& Bodys):**

- OK - 200 :

```
{ }
```

```

1 {
2     "id": ..., //concrete user (see api path)
3     "first_name": ...,
4     "last_name": ...,
5     "balance": ...
6 }

```

- Not Found - 404

- Bad Request - 400

- DELETE http://localhost:8080/casino/bank/api/user/{user_id}

- Request Body: nicht nötig
- Response Types (& Bodys):

- OK - 200 :

```
{ }
```

```
1 {
2     //user id deliberately not shown
3     "first_name": ...,
4     "last_name": ...,
5     "balance": ...
6 }
```

- Not Found - 404

- POST http://localhost:8080/casino/bank/api/user/{user_id}/deposit/{amount}/{decimals}

- Request Body: nicht nötig
- Response Types (& Bodys):

- OK - 200 :

```
{ }
```

```
1 {
2     "id": ..., //concrete user (see api path)
3     "first_name": ...,
4     "last_name": ...,
5     "balance": ... //balance adjusted by amount
6 }
```

- Not Found - 404

- Bad Request - 400 (wenn amount / decimals negativ oder keine Zahl oder decimals länger als 2 Stellen)

- GET http://localhost:8080/casino/bank/api/transactions

- Request Body: nicht nötig
- Response Types (& Bodys):

- Ok - 200 :

```
{ }
```

```
1 [
2     {
3         "id": ...,
4         "user_id": ..., //some user
5         "amount": ... //some positive or negative decimal
6     },
7     {
8         "id": ...,
9         "user_id": ..., //possibly another user
10        "amount": ...
11    },
12    ...
13 ] //list of all user-wide transactions
```

oder

```
{ }
```

```
1 [] //empty array (initial system state)
```

- GET `http:\\localhost:8080\\casino\\bank\\api\\transactions\\user\\{id}`

- **Request Body:** nicht nötig
- **Response Types (& Bodys):**

- **Ok - 200 :**

```
{ }
```

```
1  [
2      {
3          "id": ..., //transaction id
4          "amount": ... //some positive or negative decimal
5      },
6      {
7          "id": ...,
8          "amount": ...
9      },
10     ...
11 ] //list of transactions of concrete user (see api path)
```

oder

```
{ }
```

```
1  [] //empty array (initial system state)
```

- **Not Found - 404**

- POST `http:\\localhost:8080\\casino\\bank\\api\\transaction\\user\\{user_id}`

- **Request Body:**

```
{ }
```

```
1  {
2      "invoicing_party": ..., //some known game service name
3      "amount": ..., //some positive or negative decimal
4  }
```

- **Response Types (& Bodys):**

- **OK - 201 :**

```
{ }
```

```
1  {
2      "id": ..., //auto generated and given transaction id
3      "invoicing_party": ...,
4      "user": ..., //concrete user_id (see api path)
5      "amount": ...
6  }
```

- **Not Found - 404**

- **Bad Request - 400** (wenn *Request Body* fehlerhaft oder Rechnungssteller unbekannt)

- PUT `http:\\localhost:8080\\casino\\bank\\api\\transaction\\{transaction_id}`

- **Request Body:**

```
{ }
```

```
1  {
2      "invoicing_party": ...,
3      "user": ..., //concrete user_id (see api path)
4      "amount": ...
5  }
```

- **Response Types (& Bodys):**

- OK - 200 :

```
{ }
```

```
1 {
2     "id": ..., //auto generated and given transaction id
3     "invoicing_party": ..., //some known game service name
4     "user": ..., //concrete user_id (see api path)
5     "amount": ... //some positive or negative decimal
6 }
```

- Not Found - 404 (wenn user oder transaction_id nicht auffindbar)
- Bad Request - 400

- DELETE http:\\localhost:8080\\casino\\bank\\api\\transaction\\{transaction_id}

- **Request Body:** nicht nötig

- **Response Types (& Bodys):**

- Ok - 200 :

```
{ }
```

```
1 {
2     //transaction id deliberately not shown
3     "invoicing_party": ...,
4     "user": ...,
5     "amount": ...
6 }
```

- Not Found - 404

folgende Roulette-Service-APIs sollen über die Anwendung nach Außen bereitgestellt werden:

- POST http:\\localhost:8081\\casino\\roulette\\api\\play

- **Request Body:**

```
{ }
```

```
1 {
2     "user": ..., //some user id
3     ... //additional roulette specific domain logic
4 }
```

- **Response Types (& Bodys):**

- Ok - 200 :

```
{ }
```

```
1 {
2     "user": ..., //some user id
3     "winning": ..., //bool
4     "amount": ..., //positive or negative decimal
5     "ball_position": ..., //landing position of the ball
6     ... //additional information of the users initial bet setup
7     ... //additional roulette specific domain logic
8 }
```

- Not Found - 404
- Bad Request - 400

- GET http:\\localhost:8081\\casino\\roulette\\api\\info\\rules

- **Request Body:** nicht nötig

- **Response Types (& Bodys):**

- Ok - 200 :



```
1 "some text explaing the game rules and betting options"
```

- GET `http:\\localhost:8081\\casino\\roulette\\api\\info\\chances`

- **Request Body:** nicht nötig

- **Response Types (& Bodys):**

- Ok - 200 :



```
1 "some text showing a table with win chances
2 and formulas for calculating profits"
```

- GET `http:\\localhost:8081\\casino\\roulette\\api\\stats`

- **Request Body:** nicht nötig

- **Response Types (& Bodys):**

- Ok - 200 :

```
{ }
```

```
1 {
2     "total_client_count": ..., //amount of diffrent clients
3     "total_games_count": ..., //amount of all played games
4     "total_profit": ...,
5     "total_cash_out": ...,
6     "total_turnover": ...
7 }
```

- GET `http:\\localhost:8081\\casino\\roulette\\api\\stats\\user\\{user_id}`

- **Request Body:** nicht nötig

- **Response Types (& Bodys):**

- Ok - 200 :

```
{ }
```

```
1 {
2     "client": ..., //concrete client id (see api path)
3     "total_games_count": ...,
4     "total_winnings": ...,
5     "total_losses": ...,
6     "total_client_profit": ...,
7     "total_house_turnover_from_client": ...,
8     "total_house_profit_from_client": ...
9 }
```

- Not Found - 404

- GET `http:\\localhost:8081\\casino\\roulette\\api\\stats\\games`

- **Request Body:** nicht nötig

- **Response Types (& Bodys):**

- Ok - 200 :

```
{ }
```



```

1  [
2      {
3          id: ..., //game/stat-entry id
4          user: ..., //user id
5          "winning": ..., //bool
6          "amount": ..., //positive or negative decimal
7          "ball_position": ..., //landing position of the ball
8          ... //additional information of the users initial bet setup
9          ... //additional roulette specific domain logic
10     },
11     {
12         id: ...,
13         user: ...,
14         "winning": ...,
15         "amount": ...,
16         "ball_position": ...,
17         ... //additional information of the users initial bet setup
18         ... //additional roulette specific domain logic
19     },
20     ....
21 ]

```

oder

```
{ }
```

```
1  [] //empty array (initial system state)
```

- GET `http://localhost:8081/casino/roulette/api/stat/{game_id}`

- **Request Body:** nicht nötig
- **Response Types (& Bodys):**

- **Ok - 200 :**

```
{ }
```

```

1  {
2      id: ..., //game/stat-entry id
3      user: ..., //user id
4      "winning": ..., //bool
5      "amount": ..., //positive or negative decimal
6      "ball_position": ..., //landing position of the ball
7      ... //additional information of the users initial bet setup
8      ... //additional roulette specific domain logic
9  },

```

- **Not Found - 404**

- DELETE `http://localhost:8081/casino/roulette/api/stat/{game_id}`

- **Request Body:** nicht nötig
- **Response Types (& Bodys):**

- **Ok - 200 :**

```
{ }
```

```

1  {
2      id: ..., //game/stat-entry id
3      user: ..., //user id
4      "winning": ..., //bool
5      "amount": ..., //positive or negative decimal
6      "ball_position": ..., //landing position of the ball
7      ... //additional information of the users initial bet setup
8      ... //additional roulette specific domain logic

```

```
9 }
```

- Not Found - 404

folgende Slotmachine-Service-APIs sollen über die Anwendung nach Außen bereitgestellt werden:

- POST http:\\localhost:8081\\casino\\slots\\api\\play

- **Request Body:**

```
{ }  
  
1 {  
2     "user": ..., //some user id  
3     ... //additional slotmachine specific domain logic  
4 }
```

- **Response Types (& Bodys):**

- Ok - 200 :

```
{ }  
  
1 {  
2     "user": ..., //some user id  
3     "winning": ..., //bool  
4     "amount": ..., //positive or negative decimal  
5     "slot_states": [...], //landing symbol of the slots  
6     ... //additional information of the users initial bet setup  
7     ... //additional slotmachine specific domain logic  
8 }
```

- Not Found - 404
- Bad Request - 400

- GET http:\\localhost:8081\\casino\\slots\\api\\info\\rules

- **Request Body:** nicht nötig

- **Response Types (& Bodys):**

- Ok - 200 :



```
1 "some text explaing the game rules and betting options"
```

- GET http:\\localhost:8081\\casino\\slots\\api\\info\\chances

- **Request Body:** nicht nötig

- **Response Types (& Bodys):**

- Ok - 200 :



```
1 "some text showing a table with win chances  
2 and formulas for calculating profits"
```

- GET http:\\localhost:8081\\casino\\slots\\api\\stats

- **Request Body:** nicht nötig

- **Response Types (& Bodys):**

- Ok - 200 :

```
{ }
```

```

1  {
2      "total_client_count": ..., //amount of different clients
3      "total_games_count": ..., //amount of all played games
4      "total_profit": ...,
5      "total_cash_out": ...,
6      "total_turnover": ...
7  }

```

- GET `http://localhost:8081/casino/slots/api/stats/user/{user_id}`

- **Request Body:** nicht nötig
- **Response Types (& Bodys):**

- Ok - 200 :

```
{ }
```

```

1  {
2      "client": ..., //concrete client id (see api path)
3      "total_games_count": ...,
4      "total_winnings": ...,
5      "total_losses": ...,
6      "total_client_profit": ...,
7      "total_house_turnover_from_client": ...,
8      "total_house_profit_from_client": ...
9  }

```

- Not Found - 404

- GET `http://localhost:8081/casino/slots/api/stats/games`

- **Request Body:** nicht nötig
- **Response Types (& Bodys):**

- Ok - 200 :

```
{ }
```

```

1  [
2      {
3          id: ..., //game/stat-entry id
4          user: ..., //user id
5          "winning": ..., //bool
6          "amount": ..., //positive or negative decimal
7          "slot_states": [...], //landing symbol of the slots
8          ... //additional information of the users initial bet setup
9          ... //additional slotmachine specific domain logic
10     },
11     {
12         id: ...,
13         user: ...,
14         "winning": ...,
15         "amount": ...,
16         "slot_states": [...], //landing symbol of the slots
17         ... //additional information of the users initial bet setup
18         ... //additional slotmachine specific domain logic
19     },
20     ....
21 ]

```

oder

```
{ }
```

```
1  [] //empty array (initial system state)
```

- GET `http:\\localhost:8081\\casino\\slots\\api\\stat\\{game_id}`

- **Request Body:** nicht nötig
- **Response Types (& Bodys):**

- **Ok - 200 :**

```
{ }
```

```
1 {
2     id: ..., //game/stat-entry id
3     user: ..., //user id
4     "winning": ..., //bool
5     "amount": ..., //positive or negative decimal
6     "slot_states": [...], //landing symbol of the slots
7     ... //additional information of the users initial bet setup
8     ... //additional slotmachine specific domain logic
9 }
```

- **Not Found - 404**

- DELETE `http:\\localhost:8081\\casino\\slots\\api\\stat\\{game_id}`

- **Request Body:** nicht nötig
- **Response Types (& Bodys):**

- **Ok - 200 :**

```
{ }
```

```
1 {
2     id: ..., //game/stat-entry id
3     user: ..., //user id
4     "winning": ..., //bool
5     "amount": ..., //positive or negative decimal
6     "slot_states": [...], //landing symbol of the slots
7     ... //additional information of the users initial bet setup
8     ... //additional slotmachine specific domain logic
9 }
```

- **Not Found - 404**

Tipp:

In Micro-Service-Architekturen (Systemarchitektur) werden komplexe Probleme durch die Zusammenarbeit einzelner Services gelöst. Micro-Service dürfen und sollten zum Lösen geeigneter Probleme kooperieren und gegenseitige Abfragen stellen.

Tipp:

Bei einer Vertikal-Slice-Architektur (Banking-Service) ist es Komponenten verschiedener Sub-Domains untersagt miteinander zu kommunizieren, selbst wenn diese Teil des selben logischen Layers sind.

In Fällen wo Sub-Domains übergreifende Logik notwendig ist, ist ein Lösungsansatz die Anwendung einen externen Call (`http`) gegen sich selbst machen zu lassen, so als wenn eine externe API die benötigte Logik der anderen Sub-Domäne bereitstellen würde.

Nicht-Funktionale Anforderungen

Achtung:

Nicht-funktionale Anforderungen gewährleisten die Qualität der ausgelieferten Software. Qualität ist das Ziel dieser Arbeiten und ist ausschlaggebend für die Bewertung.

Nicht-funktionalen Anforderungen dürfen nicht vernachlässigt werden!

folgende nicht-funktionale Anforderungen müssen erfüllt sein:

- die Anwendung muss Lauffähig sein
- die Anwendung muss ohne die Notwendigkeit lokale Dependencies installieren zu müssen (mit Hilfe von Docker + Maven) baubar sein
- die einzelnen Services folgen dem MVC Muster und der Vertikal-Slice-Architektur
- die Spiele-Services folgen der Geschichteten Architektur
- der Banking-Service folgt der Vertikal-Slice-Architektur
- jeder Service arbeitet mit der eigenen Datenbank
- die zwei Spielservices kommunizieren mit dem Banking-Service für den Zweck der User-Verifizierung und der Kontostandverwaltung
- SwaggerUI muss für jeden der 3 Kern-Services bereitgestellt werden
- jeglicher Code muss weitestgehend den SOLID-Prinzipien folgen
- Domain-Model-Klassen müssen weitestgehend robust sein (Instanzen dürfen zu keinem Zeitpunkt ihrer Existenz die Domainintegrität verletzen oder verletzen können)
- jeder der Services muss ausgiebig mit Hilfe von Unit-Tests geprüft werden (Path-Coverage, Blackbox-Testing, Test-First-Development)
- jede Klasse wird mit einer designierten Test-Klasse getestet
- jegliche Unit-Tests müssen weitestgehend randomisiert sein, volle Wertebereiche und Extremwerte abdecken
- alle abstrakten Dependencies einer getesteten Klasse sollen durch Mocking wegabstrahiert werden (mit Hilfe von Mockito)
- die Code-Abgabe muss mit einer "README"-Datei ausgeliefert werden die Projektbeschreibung, Nutzungsanweisung, Installationanleitung, Autorenangaben und eine Creative-Commons-Lizenz enthält (Lizenz optional)
- alle Einträge in jeglichen Datenbanken benutzen long s als ID
- alle Geldbeträge sollten in betroffenen Modellen mit BigDecimal dargestellt werden
- Spring Boots Entity Framework und Dependency Injection müssen weitestgehend genutzt werden

Info:

Manchmal ist es nicht Möglich an gewissen Stellen alle Anforderungen auf Grund von Widersprüchen oder technischen Koplikationen einzuhalten.

Wenn Sie das Gefühl haben auf so eine Stelle gestoßen zu sein, dann lassen sich beraten und/oder dokumentieren und argumentieren Sie die als Maßnahmen getroffenen Design-Entscheidungen. Gute Argumentation (widersprüchliche Design-Prinzipien, Verstoß gegen Konventionen, Best Practices oder Framework-Vorgaben), resultiert in keinem Punkteverlust.

Lassen Sie sich regelmäßig helfen, beraten und Feedback geben!

Bewertungskriterien

- 10 Punkte für Präsentation
 - 15 Punkte für sauberes Unit-Testing
 - 15 Punkte für saubere Architektur
 - 10 Punkte für Lauffähigkeit
 - 30 Punkte für das Einhalten der Solid Prinzipien
 - 30 für das Erfüllen der Funktionalen Anforderungen
-
-